

Module 5: Software Assurance and Security Hardening

Grad Café Analytics · Luke Ellman · EN.605.256

This report documents the Module 5 hardening of the Grad Café analytics service: how to install and run it reproducibly, what the dependency graph shows, how the SQL layer defends against injection, how row limits are enforced, how the database account is restricted to least privilege, why the project is packaged, and how continuous integration enforces all of it on every push.

1. Installing and running the project

The project installs two ways. Both start from a clean virtual environment, and both finish with an editable install so that `import src.query_data` resolves identically from the application, the tests, Sphinx and CI.

Option A — pip + venv

```
cd module_5
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
```

Option B — uv

```
cd module_5
uv venv
source .venv/bin/activate
uv pip sync requirements.lock
uv pip install -e .
```

The two are not equivalent, and the difference matters in practice. `pip install -r` adds what is listed, resolves transitive dependencies, and leaves anything already present in place. `uv pip sync` makes the environment *match* the file, removing packages that are not in it — but because it installs exactly what is listed, it needs a fully resolved lockfile rather than the abstract `requirements.txt`. Pointing it at the abstract file produces a subtly broken environment: `pytest` installs without `pluggy` and fails on import. `requirements.lock` is the resolved file, generated with `uv pip compile requirements.txt -o requirements.lock`, and both install paths were verified from an empty virtual environment with the full test suite.

`requirements.txt` carries the runtime dependencies (Flask, psycopg, SQLAlchemy, BeautifulSoup, Selenium), the test tooling (pytest, pytest-cov), the Module 5 analysis tooling (pylint, pydeps) and Sphinx. Running the application also needs PostgreSQL 14+, and regenerating the dependency graph needs Graphviz, since `pydeps` shells out to `dot`.

2. Packaging: why setup.py matters

Without packaging, whether `import src.db` works depends on which directory the interpreter happened to start in, which is the classic source of "works on my machine". `setup.py` declares the project as a real distribution, so an editable install (`pip install -e .`) puts it on the path once and every entry point agrees thereafter.

Concretely it buys four things:

- **Stable imports.** Tests, Sphinx autodoc and the Flask app all resolve `src.*` the same way, with no `sys.path` shims.
- **Declared dependencies.** `install_requires` states the runtime set, and the dev extra separates tooling, so a production install stays lean.
- **Console scripts.** `gradcafe-load`, `gradcafe-report` and `gradcafe-scrape` become real commands.
- **Tool interoperability.** `uv` and `pip` can both read the metadata when resolving an environment.

3. Dependency graph

`dependency.svg` was generated with `python -m pydeps src --noshow -T svg -o dependency.svg --cluster --max-bacon=2`, which walks the import graph and renders it through Graphviz.

The graph shows ten first-party modules and five external packages, arranged in a strict layering with no cycles. At the base sits `src.db`, the single place a connection is opened; `psycpg` feeds it, and everything that touches the database depends on it. `src.load_data` builds on `src.db` to compose the INSERT and own the schema, and `src.query_data` depends on both `src.db` and `src.load_data`, the latter because it derives its column allow-list from the table definition rather than repeating it. The ETL pair `src.scrape` (which pulls in BeautifulSoup and Selenium) and `src.clean` both depend only on `src.fields`, the shared field-name tuple, which keeps the producer and consumer of the raw record shape from drifting apart. SQLAlchemy enters through `src.models`, which `src.orm_queries` uses for the ORM cross-check of the raw SQL. Everything converges on `src.flask_app`, which depends on Flask plus the scraper, loader and query modules it injects as services, and on which nothing else depends. That shape is the point: the web layer is replaceable, the data layer has no knowledge of HTTP, and the single database entry point is what makes the credential and SQL-composition rules in section 4 enforceable in one place.

4. SQL injection defenses

Module 4 built SQL with Python f-strings. Every statement in `src/` is now a composed `psycpg.sql` object, constructed separately from the values it binds. There is no f-string, concatenation or `.format()` applied to SQL text anywhere in the package.

Risk	Defense	Where
Values spliced into SQL text	Every value bound with <code>sql.Placeholder()</code> and passed to <code>sql.execute(statement, params)</code>	<code>src.load_data.build_row_sql(state.params)</code> and <code>src.query_data.build_query_sql(state.params)</code>
Identifiers spliced into SQL text	Table and column names wrapped in <code>sql.Identifier.quote()</code> and used in <code>src.load_data.INSERT_ROW_SQL</code>	<code>src.load_data.INSERT_ROW_SQL</code>
User-chosen sort column	Checked against the <code>ROW_KEYS</code> allow-list before composing <code>sql.order_by</code> clauses	<code>src.query_data.order_by_clause</code>
Unbounded result sets	Every statement ends in <code>LIMIT %s</code> ; <code>clamp_limit()</code> forces data to <code>clamp_limit</code>	<code>src.query_data.clamp_limit</code>
Wildcard abuse ('%' returning everything)	<code>sql.Identifier.quote()</code> escapes <code>%</code> , <code>_</code> and backslash so the query database treats it like	<code>src.query_data.quote_identifier</code>

The composition reads as follows, with construction and execution on separate lines:

```

statement = sql.SQL("SELECT {cols} FROM {table}").format(
    cols=sql.SQL(", ").join(sql.Identifier(k) for k in ROW_KEYS),
    table=sql.Identifier(TABLE_NAME),
)
matches = sql.SQL(" OR ").join(
    sql.SQL("{col} ILIKE {val}").format(
        col=sql.Identifier(name), val=sql.Placeholder()
    ) for name in SEARCHABLE_COLUMNS
)
# ... execution happens separately, with the values as parameters:
cur.execute(statement, params + (clamp_limit(limit),))

```

Why that is safe: the parameters never pass through the SQL parser. `psycpg` sends the statement and the values separately, so a value can only ever be compared as data. The strongest demonstration in the test suite is `test_the_sql_text_is_identical_whatever_the_user_sends`, which renders the composed statement for six injection payloads (`' ; DROP TABLE applicants; --, ' OR '1'='1`, a UNION SELECT, and the LIKE metacharacters) and asserts the SQL text is byte-for-byte identical in every case. Only the bound parameters differ. A companion test confirms the tautology `' OR '1'='1` returns zero rows, because it is matched as a literal string, and that the table still holds exactly the rows it started with afterwards.

5. LIMIT enforcement

Every statement the application issues carries a LIMIT, including the single-row aggregates, which are capped at 1. The limit is never taken from the request as given: `clamp_limit()` parses it and forces the result into 1..100.

Request	Effective limit	Why
limit=25	25	inside the allowed range
limit=0 or -5	1	clamped up to MIN_LIMIT
limit=101 or 1000000	100	clamped down to MAX_LIMIT
limit=abc, 3.5, missing	10	unparseable, falls back to the default
limit=10; DROP TABLE applicants	10	unparseable, falls back to the default

The clamp is applied at the boundary, in `flask_app.read_row_options()`, and again inside `get_recent_rows()`, so a caller that bypasses the web layer is bounded too. The rendered form echoes the clamped value, not the requested one, so an oversized request is visibly corrected rather than silently ignored.

6. Least-privilege database configuration

No credential appears in the source. `src/db.py` reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` and `DB_PASSWORD` from the environment, falling back to a `DATABASE_URL` string. `.env.example` lists the names with placeholders; `.env` is in `.gitignore` and is never committed. `describe_connection()` redacts the password so it cannot reach a log line.

The application role was created by `db/least_privilege.sql`:

```

CREATE ROLE gradcafe_app
LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOBYPASSRLS

```

```

PASSWORD :app_password;

GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
REVOKE CREATE ON SCHEMA public FROM gradcafe_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    REVOKE ALL ON TABLES FROM gradcafe_app;

```

What was granted and why. The application does exactly two things to the database: it reads applicants for the analysis page, and it appends newly scraped rows on POST /pull-data. So it gets SELECT and INSERT on that one table, and nothing else. It never updates or deletes a row, so UPDATE and DELETE are withheld; it never changes the schema, so CREATE is revoked from the schema and the role is NOCREATEDB and NOCREATEROLE; it has no reason to read another database's catalogue, so it is NOSUPERUSER and NOBYPASSRLS. The default-privileges revoke means a table added later is inaccessible until an administrator deliberately grants access, so the blast radius does not grow quietly as the schema does.

Schema creation was moved out of the request path to make this work honestly. CREATE TABLE IF NOT EXISTS requires CREATE on the schema even when the table already exists, so ensure_schema() now checks to_regclass() first and issues DDL only when the table is genuinely missing. In normal operation the application sends no DDL at all; creating the table is an administrator task.

Verified behaviour under the restricted role:

Operation	Result
GET /analysis and every reporting query	works (HTTP 200)
POST /pull-data inserting new rows	works
DROP TABLE applicants	denied: InsufficientPrivilege
ALTER TABLE applicants ADD COLUMN	denied: InsufficientPrivilege
CREATE TABLE evil	denied: InsufficientPrivilege
DELETE FROM applicants	denied: InsufficientPrivilege
UPDATE applicants SET status = 'x'	denied: InsufficientPrivilege
SELECT rolpassword FROM pg_authid	denied: InsufficientPrivilege

That is the payoff from sections 4 and 6 together: parameterisation means an injection attempt is treated as data, and least privilege means that even if a statement did somehow escape, the account executing it cannot drop the table, modify a row, or read the password hashes.

7. Static analysis and continuous integration

python -m pylint src scores **10.00/10**; the captured run is in pylint_report.txt. Reaching it was mostly real repair rather than suppression: the duplicated field tuples moved into src/fields.py, ROW_KEYS is now derived from the table definition, create_app takes a services mapping instead of seven positional arguments, the clause builders carry their own parameters, and func.count was replaced with the concrete sqlalchemy.sql.functions.count class. Three .pylintrc settings and one inline

disable remain, each commented in place and each covering a documented false positive.

`.github/workflows/ci.yml` runs four jobs on every push and pull request:

Job	What it enforces
pylint	python -m pylint src --fail-under=10; the build fails below 10.00
dependency-graph	installs Graphviz, regenerates dependency.svg with pydeps, and fails if the file is missing, empty, or no longer reflects
snyk	snyk test against requirements.txt for known vulnerable dependencies
pytest	starts PostgreSQL 16 and runs the full marked suite; pytest.ini enforces 100% coverage of module_5/src, so a coverage

The test suite grew to 291 tests, all marked, at 100% statement coverage. `tests/test_sql_safety.py` is new in Module 5 and covers the hardening directly: the limit clamp across twelve inputs, the identifier allow-list against every injection payload, proof that the composed SQL text does not vary with user input, the LIKE-wildcard escaping, the endpoint's handling of hostile query strings, and the environment-driven credential handling including password redaction.

8. Supply-chain and static application security testing

Dependency scan. `snyk test --file=requirements.txt --package-manager=pip` tested **59 dependencies** and found **no vulnerable paths** (see `snyk-analysis.png`). Nothing needed patching or removing. The same scan runs in CI, where a missing `SNYK_TOKEN` emits a warning rather than passing silently, so an unconfigured secret cannot be mistaken for a clean result.

Static analysis (extra credit). `snyk code test` reported six findings, all LOW, with no HIGH or MEDIUM. One was fixed and five were triaged as not exploitable:

Finding	Location	Disposition
Use of Hardcoded Passwords	<code>tests/test_sql_safety.py</code>	FIXED. The rule fires on the shape <code>{"password": <literal>}</code> , not on the value, so the assumption
Path Traversal (x2)	<code>src/clean.py</code> lines 115, 120	ACCEPTED. <code>--in/--out</code> come from the operator's own command line.
Path Traversal (x2)	<code>src/scrape.py</code> lines 222, 228	ACCEPTED. <code>--out</code> comes from the operator's own command line.
Path Traversal (x1)	<code>src/load_data.py</code> line 364	MITIGATED. <code>GRADCAFE_DATA_FILE</code> is now expanded, resolved and checked with <code>os.path</code>

Why the five are accepted. Every one is a command-line entry point or an environment variable read by a tool the operator runs themselves. The input is already under the control of whoever starts the process, so no privilege or trust boundary is crossed: someone able to pass `--out ../etc/something` can equally run `cat` directly. This is the opposite of the web layer, where input genuinely arrives from an untrusted client and is therefore parameterised, allow-listed and clamped as described in sections 4 and 5. Recording the reasoning matters more than the count: a LOW finding that is understood and accepted is a decision, whereas one that is silenced is a liability.